AGENTIC CI

Foundations, Architectures & Practice

Technical Notes 10 & 11 — with Easy Examples

Prof. César Moreno Pascual PhD.

Corporate Intelligence Course | Universidad Francisco de Vitoria
2025-2026 Academic Year

Course Roadmap

TN10

AGENTIC FOUNDATIONS

LLMs, Autonomy, 4 Design Patterns, ReAct, RAG, Embeddings, ChromaDB, Tools, Multi-Agent

TN11

INTEGRATION & PRACTICE

Hallucination, Self-Consistency, Reflexion, Confidence Scoring, LangGraph, 10-Stage Pipeline, Claude Code Lab

PART I: TN10

Agentic Foundations & Multi-Agent Architectures

Key Definitions: LLMs & Agentic AI

Large Language Model (LLM)

A neural network trained on billions of sentences that can understand and generate human language. Examples: Claude (Anthropic), GPT-4 (OpenAI). LLMs serve as the cognitive engine of agentic CI systems — they reason about competitive dynamics, but need tools and frameworks to avoid hallucination.

Agentic AI

An AI workflow where the LLM executes multiple steps to complete a task — rather than generating a single output from a single prompt. The LLM iterates, reflects, uses tools, and adapts. Andrew Ng showed that wrapping GPT-3.5 in an agentic workflow outperformed GPT-4 used non-agentically.

Why 'Agentic' Matters for CI

Your CI frameworks (AMC, Exchange Value, ECOMO, War Gaming, Blind Spots) require multi-step analytical workflows. Agentic AI can autonomously execute these workflows — collecting OSINT data, scoring competitors, generating scenarios, and producing executive briefs with citations.

Why Do We Need All of This?

LLMs are powerful, but they have critical limitations for CI — each one drives an architectural choice

LLM Limitation	CI Risk	Solution (covered in TN10/TN11)
Knowledge Cutoff	Missing recent competitor actions, regulatory changes	**RAG + Real-time Tool Use**
Hallucination	Fabricated revenue figures, non-existent patents, wrong executive names	**Anti-hallucination patterns (TN11)**
Context Length	Cannot process entire 10-K filings in one pass	**Document chunking in RAG**
Reasoning Depth	Complex multi-step AMC analysis may fail in one shot	**ReAct iterative loops**
Source Attribution	Claims without verifiable sources undermine CI credibility	**Mandatory citation architecture (TN11)**

💡 **Easy Example: A Student Writing a Research Paper**

Knowledge Cutoff = using a 2-year-old textbook → RAG gives them today's journal articles
Hallucination = misremembering a statistic → Source verification makes them check every claim
Reasoning Depth = trying to solve a complex problem in one step → ReAct breaks it into sub-steps

The Agentic Autonomy Spectrum

Increasing Autonomy →

Less Autonomous	Semi-Autonomous	Highly Autonomous
Predetermined steps Hard-coded tool calls *CI: Scheduled pricing scrape*	LLM chooses from predefined tools *CI: AMC agent picks data sources*	LLM decides sequence Creates tools dynamically *CI: Full CI pipeline:* *detect → collect → decide*
🍽️ Microwave meal Same steps every time	📦 HelloFresh kit Guide + your decisions	👩‍🍳 Professional chef Assess, plan, adapt, create

CI → Your frameworks (AMC, EV, ECOMO, War Gaming, Blind Spots) provide the analytical structure.
Agentic AI provides the computational engine to execute them — from Level 1 to Level 3.

Four Design Patterns for CI Agents

Andrew Ng's framework — each maps to a CI framework you already know

Reflection

Agent critiques its own output

CI → Devil's Advocacy (TN03)

💡 Re-reading your own essay and fixing weak arguments

Tool Use

Agent calls external APIs

CI → OSINT Collection (TN03)

💡 Using a calculator during a maths exam instead of guessing

Planning

Agent decomposes complex tasks

CI → CI Workflow (TN01)

💡 Planning a holiday: flights → hotels → transport → activities

Multi-Agent

Specialised agents collaborate

CI → War Gaming roles (TN02)

💡 Hospital surgery team: surgeon, anaesthetist, nurse, coordinator

Key Definitions: ReAct, RAG & Embeddings

ReAct (Reasoning + Acting)

Yao et al. (2023). A framework where the LLM interleaves Thought (reasoning about what to do), Action (tool call — web search, API, database), and Observation (result fed back). The loop repeats until the agent has enough information. This is how the CI agent autonomously walks through the analytical workflow.

RAG (Retrieval-Augmented Generation)

Lewis et al. (2020). Instead of answering from memory (which hallucinates), the LLM first retrieves relevant documents from a database, then generates an answer grounded in those documents. Three steps: INDEX documents → RETRIEVE the most relevant chunks → GENERATE a cited answer.

Embedding

A list of ~1,500 numbers (a 'vector') that captures the meaning of a text. An embedding model converts 'Palantir DoD contract' and 'defence AI spending' into similar numbers — even though they share zero words — because their meanings are related. This is what makes RAG possible: search by meaning, not keywords.

Vector Database (ChromaDB)

A database that stores embeddings and lets you search by mathematical distance (meaning similarity) rather than keyword match. ChromaDB holds four CI collections: competitor_intel, market_trends, internal_strategy, and historical_archive — updated daily/weekly/monthly.

The ReAct Framework

THOUGHT

Verbal reasoning
about what to do

→

ACTION

Tool call: search,
API, database

→

OBSERVATION

Result fed back
into reasoning

← Iterate until final answer →

💡 Easy Example: Choosing What Laptop to Buy

THOUGHT: 'I need to find laptops under €1000 with good reviews.'
ACTION: You Google 'best laptops under 1000 euros 2025.'
OBSERVATION: Top results: ThinkPad X1, Dell XPS 13, MacBook Air.
THOUGHT: 'Now I need to compare battery life and weight.'
ACTION: You search for detailed specs of each.
OBSERVATION: ThinkPad 15h battery, 1.2kg; Dell 12h, 1.3kg... → Keep looping until you decide.

CI version: Same loop but asking 'Should we enter the EU EV battery market?'
using EDGAR for financials, PatentsView for R&D intel, USAspending for government contracts.
Maps to TN01 Strategic Analysis Workflow (Figure 5.1).

RAG: Retrieval-Augmented Generation

1

INDEX

Chunk documents
→ Vector embeddings
→ Store in ChromaDB

→

2

RETRIEVE

Query → embedding
→ Find top-k similar
chunks (ANN search)

→

3

GENERATE

Chunks as context
→ LLM generates
grounded + cited answer

💡 **Easy Example: Open-Book vs Closed-Book Exam**

Without RAG (closed-book): Student answers from memory → might remember wrong facts = hallucination.
With RAG (open-book): Student looks up notes first → finds exact revenue figure, correct patent date.

CI: The agent stores competitor filings in ChromaDB. When asked 'What is Palantir's government revenue trend?', it retrieves the actual 10-K filing data BEFORE generating its AMC analysis.
Result: grounded, accurate, citable intelligence — not hallucinated guesses.

Deep Dive: What Is an Embedding?

The Problem

A computer sees text as characters.
'Palantir DoD contract' and 'defence AI spending'
share ZERO words — a keyword search
would never connect them.
But they're about the same topic!

The Solution: Embeddings

Convert text into ~1,500 numbers (vector).

```
'Palantir DoD contract' → [0.82, 0.91, 0.03...]
'defence AI spending'   → [0.79, 0.88, 0.05...]
'pasta recipe'          → [0.12, 0.08, 0.94...]
```

Similar meaning → similar numbers → CLOSE in space

Unrelated meaning → different numbers → FAR apart

Palantir ↔ Defence AI → Distance: 0.05 (very close!)

Palantir ↔ Pasta recipe → Distance: 0.89 (very far!)

💡 Easy Example: The Meaning-Based Library

Imagine a library where books are shelved by TOPIC SIMILARITY, not alphabetically.
'Military AI contracts' sits next to 'Palantir defence revenue' — zero shared words, same topic.
When you ask about 'DoD technology spending', the librarian points you straight to that shelf.
That librarian = the embedding model. Those shelves = ChromaDB.

Why ChromaDB? Vector DB vs Regular DB

Regular Database (SQL, Excel)

- Searches by exact keyword match
- 'defence AI spending' → finds ONLY those exact words
- Misses semantically related documents
- Like searching a library by title only

Vector Database (ChromaDB)

- Searches by mathematical distance (meaning)
- 'defence AI spending' → ALSO finds 'Palantir DoD contract'!
- Discovers related docs regardless of wording
- Like a librarian who understands topics

ChromaDB CI Collections

`competitor_intel`
Updated: Daily

`market_trends`
Updated: Weekly

`internal_strategy`
Updated: Monthly

`historical_archive`
Updated: Continuous

CI → This is what eliminates hallucination: the agent always retrieves current, verified OSINT data before answering — never guesses from training memory.

Multi-Agent Architecture & Tool Use

Collector
(Intel Officer)

AMC Analyst
(Control Team)

Orchestrator
(Exercise Director)

Scenario Planner
(Game Master)

Verifier
(Red Team)

💡 **Easy Example: A Formula 1 Pit Stop + Investigative Journalist**

Pit Stop: Wheel changers, jack operators, fuel handlers — each specialised, working in parallel, coordinated by the crew chief. If one fails, others continue (error isolation).

Journalist: Doesn't write from memory — calls sources, checks public records, financial databases.
Agent does the same: USAspending for contracts, EDGAR for financials, PatentsView for R&D.
Each tool maps to an AMC dimension: USAspending → Capability | PatentsView → Awareness

CI → War Gaming (TN02): Blue Team → Strategy Agent | Red Team → Competitor Agents (each with AMC profile) | Control → Orchestrator | Debrief → Reporter. Days → Minutes.

PART II: TN11

Hallucination, Framework Integration & Agentic CI Practice

Key Definitions: Hallucination & Countermeasures

Hallucination

When an AI generates plausible but false information. Three types: Factual (wrong numbers), Contextual (misattributed info), Logical (flawed reasoning). In CI, hallucinated revenue figures or fabricated patent data can lead to million-euro strategic errors.

Self-Consistency (Wang et al., 2023)

Generate 3-5 independent reasoning paths for the same question, then select the most consistent answer. If all paths agree → HIGH confidence. If they diverge → flag for human review. Maps to Dialectical Inquiry (TN03).

Reflexion (Shinn et al., 2023)

The agent reflects on its own failures and adjusts its approach: Generate → Evaluate → Reflect on errors → Retry with improved strategy. The agent learns what types of mistakes it makes. Maps to Pre-Mortem Analysis (TN03) and the Learning Cycle (TN01).

Confidence Scoring (HIGH/MEDIUM/LOW)

Every finding gets a confidence level. HIGH = 2+ authoritative sources agree. MEDIUM = single reliable source. LOW = inference only. UNCERTAIN = conflicting data. LOW findings are flagged for human review before inclusion in reports.

Three Types of Hallucination in CI

CRITICAL

FACTUAL

Revenue: $2.1B
(actual: $1.4B)

💡 Misremembering an exam answer

Fix → Source verification + RAG

HIGH

CONTEXTUAL

CEO quote → wrong company

💡 Citing the wrong book

Fix → Citation architecture

HIGH

LOGICAL

'Safe: 1 exited'
(ignoring 3 new!)

💡 A flawed syllogism

Fix → Self-consistency checks

CI → Hallucination = technological Blind Spots (TN03).
Devil's Advocacy → Reflection | Dialectical Inquiry → Multi-agent debate | Pre-Mortem → Self-consistency

Strategy 1: Self-Consistency

CI Question: Competitor Motivation?

Path A → M = HIGH
Revenue concentration

Path B → M = HIGH
CEO statements

Path C → M = MEDIUM
Diversification

Majority Vote → M = HIGH (2/3 agree)

💡 Easy Example: Asking Three Independent Financial Analysts

You ask 3 analysts to independently assess a competitor's motivation:
Analyst A (revenue data) → HIGH | Analyst B (CEO statements) → HIGH | Analyst C (portfolio) → MEDIUM

2 out of 3 agree on HIGH → much higher confidence than one analyst.
The disagreement from Analyst C is itself valuable — it tells you WHERE the uncertainty lies.
Divergence flags the finding for human review (maps to Dialectical Inquiry, TN03).

Strategy 2: Reflexion

1. Generate → 2. Evaluate → 3. Reflect → 4. Retry

← Iterate until quality threshold →

💡 Easy Example: Practising a Presentation

1. Generate: You give the talk for the first time.
2. Evaluate: You record yourself and watch the playback.
3. Reflect: 'I talked too fast, skipped financial evidence, didn't address the competitor.'
4. Retry: Practise again — slower, with data slides, addressing the gap.

CI version: Agent generates AMC scores for Palantir.
Evaluator: 'You scored Capability=5 but cited no financial data.'
Agent reflects → retrieves 10-K filing → re-scores with actual revenue and R&D data.

Maps to the Learning Cycle (TN01 Fig 5.1) and Pre-Mortem Analysis (TN03).
The agent builds a memory of what types of errors it makes, improving over time.

Key Definitions: LangGraph & the CI Pipeline

LangGraph

A framework that extends LangChain with graph-based workflows. Five features: Nodes (agents/functions), Edges (transitions), State (persistent data), Conditionals (branch based on results), and Cycles (iterative refinement). Think of it as the 'assembly line blueprint' that wires all CI agents together.

Human-in-the-Loop

For high-stakes Go/No-Go decisions, the LangGraph pipeline pauses and presents findings for human approval before generating the final report. The AI accelerates analysis; humans make the final call. Required for EU AI Act 'High Risk' classification.

The 10-Stage Agentic CI Pipeline

The complete architecture mapping every CI framework to an agent node: 1. Trigger (TN03) → 2. Collect (TN03) → 3. Profile AMC (TN01) → 4. Predict (TN01) → 5. Scenarios (TN02) → 6. Exchange Value (TN01) → 7. ECOMO (TN02) → 8. Verify (TN11) → 9. Bias Check (TN03) → 10. Report (TN01). With a feedback loop back to the Learning Cycle.

LangGraph: Stateful CI Orchestration

Nodes	Edges	State	Conditionals	Cycles

Collect → AMC → Scenarios → EV → ECOMO → Verify → Report

← If >30% LOW confidence → route back to Collector for more data

💡 Easy Example: Factory Assembly Line with Quality Control

Each station (node) does its specialised job and passes the product (state) to the next.
A quality inspector (conditional edge) checks output at key points.
If a product fails inspection → sent back to an earlier station for rework.

CI: If the Verifier finds too many unsourced claims, the pipeline loops back to Collector.
Only when quality thresholds are met does the report proceed to the executive.

Human-in-the-Loop: For Go/No-Go decisions, the pipeline PAUSES for human approval.
The AI accelerates the analysis — humans make the final call.

The Complete 10-Stage CI Pipeline

1	2	3	4	5
Trigger	Collect	Profile	Predict	Scenarios
TN03	TN03	TN01	TN01	TN02

6	7	8	9	10
Value	ECOMO	Verify	Bias	Report
TN01	TN02	TN11	TN03	TN01

💡 Easy Example: How a Detective Solves a Case

1. Trigger: Crime reported 2. Collect: Gather evidence 3. Profile: Build suspect profiles (AMC)
4. Predict: Predict behaviour 5. Scenarios: 3 theories 6. Value: Cost/benefit of pursuing each
7. ECOMO: Resource constraints 8. Verify: Cross-check evidence 9. Bias: Am I overlooking anyone?
10. Report: Present to prosecutor with full evidence chain
LEARNING CYCLE: After the case, outcome feeds back → improves future investigations.

War Gaming → Multi-Agent Architecture

War Game Role	Agent	System Prompt
Blue Team	Strategy Agent	Maximise your competitive position
Red Team	1 agent per competitor + AMC	You are [Competitor]. Respond?
Control	Orchestrator	Evaluate realism, inject shocks
Debrief	Reporter	What surprised us? Actions?

💡 **Easy Example: A Chess Tournament with AI Agents**

Each player = an AI agent with a different strategy (aggressive, defensive, positional).
A referee (Orchestrator) enforces rules and tracks state. After the tournament,
an analyst (Reporter) summarises which strategies worked and why.

CI: A multi-round war game that takes DAYS with humans → runs in MINUTES with agents.
Can be repeated with different assumptions to test robustness.

PART III: Hands-On Lab

Building Your CI Agent with Claude Code

Claude Code Lab: Steps 1–4

1 **Setup**

Install Claude Code, init project

2 **Architecture**

Define all 5 CI frameworks to implement

3 **OSINT Collector**

USAspending, EDGAR, PatentsView with citations

4 **AMC Analyst**

Score A/M/C (1-5) with evidence + confidence

Claude Code Lab: Steps 5–8

Universidad Francisco de Vitoria
UFV Madrid

5 **Scenarios + ECOMO**

Exchange Value + NPV → Go/No-Go

6 **Verifier + Reporter**

Anti-hallucination + executive brief with citations

7 **LangGraph**

Wire agents, conditional edges, human-in-the-loop

8 **Deploy**

React frontend → GitHub Pages

Universidad Francisco de Vitoria
UFV Madrid

AutoGen & CrewAI: Multi-Agent Frameworks

AutoGen (Microsoft)

Conversable Agents that send/receive messages.
Conversation Patterns: two-agent, group chat, hierarchical.
Human-in-the-Loop: human joins for critical decisions.

💡 Like a WhatsApp group chat: each person has a role, PM coordinates, boss joins for big decisions

CrewAI (Role-Based)

Agent = Role + Goal + Backstory + Tools.
Task = Description + Expected Output + Dependencies.
Crew = Agents + Tasks + Process (sequential/hierarchical).

💡 Like organising a film crew: director, cameraman, editor — each with skills, scenes to shoot, schedule

```
collector = Agent(role='OSINT Collector', goal='Gather competitor data', tools=[edgar, patents])
analyst = Agent(role='CI Analyst', goal='Apply AMC Framework', tools=[calculator])
crew = Crew(agents=[collector, analyst], tasks=[collect_task, analyse_task], process='sequential')
```

💡 **Key Difference**

AutoGen = conversation-driven (agents chat). CrewAI = task-driven (agents complete assigned work).

Evaluation & EU AI Act Compliance

Eval Type	Measures	Method
End-to-End	Overall pipeline quality	LLM-as-judge + human review
Hallucination Rate	Unsourced claims	Automated source-checking
Bias Detection	Blind spots (TN03)	Coverage balance check

💡 Restaurant QC

End-to-end: customer enjoyed meal?
Hallucination: menu promised what kitchen can't deliver?
Bias: always recommending expensive dishes?

EU AI Act Risk Levels

Minimal: Market monitoring → no obligations
Limited: CI chatbots → transparency required
High: Investment decisions → full documentation

💡 Easy Example: Food Safety Labels

Minimal = bottled water (basic labelling) | Limited = restaurant food (must disclose allergens)
High = baby formula (full testing, gov. approval) → If execs base M&A decisions on your CI agent = High Risk

Summary: How Everything Connects

Universidad Francisco de Vitoria
UFV Madrid

AMC Framework (TN01)	→	AMC Analyst agent
Exchange Value (TN01)	→	Exchange Value Engine
War Gaming (TN02)	→	Multi-agent debate
Scenario Planning (TN02)	→	Scenario Generator
ECOMO (TN02/TN04)	→	ECOMO Evaluator
Blind Spots (TN03)	→	Bias Check + anti-hallucination
OSINT Collection (TN03)	→	OSINT Collector + API tools

The technology serves the methodology, not the other way around.

Universidad Francisco de Vitoria
UFV Madrid

Key References

Yao et al. (2023). ReAct: Synergizing Reasoning and Acting. ICLR.

Lewis et al. (2020). Retrieval-Augmented Generation. NeurIPS.

Wu et al. (2023). AutoGen: Multi-Agent Conversation. Microsoft Research.

Wang et al. (2023). Self-Consistency Improves Chain of Thought. ICLR.

Shinn et al. (2023). Reflexion: Verbal Reinforcement Learning. NeurIPS.

Ji et al. (2023). Survey of Hallucination in NLG. ACM Computing Surveys.

Ng, A. (2025). AI Agentic Workflows. DeepLearning.AI.

Chen & Miller (2012). Competitive Dynamics. Academy of Management Annals.

Zahra & Chaples (1993). Blind Spots in Competitive Analysis. AME.